INTRODUCTION:-

This report explains a from-scratch implementation of **Multi-Scale DenseNet (MSDNet)**, a model introduced by Huang et al. in the paper **"Multi-Scale Dense Networks for Resource-Efficient Image Recognition"** (ICLR 2018).

The main goal of this model is to enable efficient image classification in situations where computational resources are limited. **MSDNet** is designed to produce predictions at multiple intermediate stages of the network, rather than only at the final layer. This allows the model to generate an earlier prediction when time or processing power is constrained. When additional computational resources are available, the network can continue processing to produce a more accurate final prediction.

This idea is particularly useful for resource-constrained platforms such as mobile phones, embedded systems, and edge devices, where executing a full deep neural network may be computationally expensive.

In this project, **MSDNet** is implemented from scratch using **PyTorch**, trained on the **CIFAR-100** dataset, and evaluated by comparing its performance with the results reported in the original research paper.

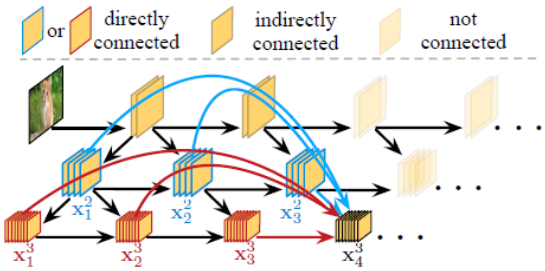
Background and Paper Summary:-

Problem Statement:-

Traditional deep neural networks perform the same amount of computation for every input, regardless of whether the input is easy or difficult to classify. MSDNet improves this by introducing **early-exit mechanisms**, which allow the model to make predictions from intermediate layers.

Because of this design, shallow layers that require less computation can be used to classify simpler inputs, while deeper layers can be used to classify more difficult inputs more accurately. This makes the model computationally efficient without sacrificing too much performance.

Architecture Overview:-



x_l^s	$\ell = 1$	$\ell = 2$	$\ell = 3$	$\ell = 4$
$s = 1$	$h_1^1(\mathrm{x}_0^1)$	$h_2^1(\mathrm{x}_1^1)$	$h_3^1(\mathrm{x}_1^1, \mathrm{x}_2^1)$	$h_4^1(\mathrm{x}_1^1, \mathrm{x}_2^1, \mathrm{x}_3^1)$
$s = 2$	$\tilde{h}_1^2(\mathrm{x}_1^1)$	$\begin{bmatrix} \tilde{h}_2^2([\mathrm{x}_1^1]) \\ h_2^2([\mathrm{x}_1^2]) \end{bmatrix}$	$\begin{bmatrix} \tilde{h}_3^2([\mathrm{x}_1^1, \mathrm{x}_2^1]) \\ h_3^2([\mathrm{x}_1^2, \mathrm{x}_2^2]) \end{bmatrix}$	$\begin{bmatrix} \tilde{h}_4^2([\mathrm{x}_1^1, \mathrm{x}_2^1, \mathrm{x}_3^1]) \\ h_4^2([\mathrm{x}_1^2, \mathrm{x}_2^2, \mathrm{x}_3^2]) \end{bmatrix}$
$s = 3$	$\tilde{h}_1^3(\mathrm{x}_1^2)$	$\begin{bmatrix} \tilde{h}_2^3([\mathrm{x}_1^2]) \\ h_2^3([\mathrm{x}_1^3]) \end{bmatrix}$	$\begin{bmatrix} \tilde{h}_3^3([\mathrm{x}_1^2, \mathrm{x}_2^2]) \\ h_3^3([\mathrm{x}_1^3, \mathrm{x}_2^3]) \end{bmatrix}$	$\begin{bmatrix} \tilde{h}_4^3([\mathrm{x}_1^2, \mathrm{x}_2^2, \mathrm{x}_3^2]) \\ h_4^3([\mathrm{x}_1^3, \mathrm{x}_2^3, \mathrm{x}_3^3]) \end{bmatrix}$

Figure: Illustration of the Multi-Scale DenseNet (MSDNet) architecture showing multi-scale feature propagation.

Three important ideas form the foundation of **MSDNet**:

1. Multi-Scale Feature Maps

In **MSDNet**, each layer processes feature maps at multiple spatial resolutions simultaneously. This design is beneficial because it allows the network to capture both **global** and **local** information even in the early stages of the architecture.

Lower-resolution feature maps provide a larger effective receptive field, enabling early classifiers to access broader contextual information. Without this mechanism, early predictions would be weaker because they would not have sufficient coverage of the overall image.

2. Dense Connections

MSDNet is inspired by the **DenseNet** architecture, in which each layer is connected to several preceding layers. Instead of discarding earlier feature representations as the network becomes deeper, the model reuses them throughout the network.

This encourages feature propagation, improves information flow, and makes optimization easier by allowing gradients to pass more effectively during training.

3. Intermediate Classifiers

Rather than producing an output only at the final layer, **MSDNet** introduces lightweight classifiers at multiple intermediate stages of the network.

During inference, the model evaluates the confidence of each intermediate prediction. If the confidence is sufficiently high, the network can terminate early and output the prediction. Otherwise, it continues processing deeper layers to generate a more accurate result.

Inference Modes:-

A key advantage of **MSDNet** is its ability to adapt inference based on the available computational budget. The model supports the following two inference modes:

Anytime Prediction:
The network is capable of producing valid predictions at multiple intermediate depths. If inference is terminated early, the prediction from the latest available classifier is returned as the output.

Budgeted Batch Classification:
Under a fixed computational budget for a batch, **MSDNet** dynamically allocates computation across samples. Simpler inputs are classified earlier, while more difficult inputs continue deeper into the network for improved accuracy.

3. Methodology

3.1 Datasets and Preprocessing

The experiments are performed on the **CIFAR-100 dataset with 100 different classes** of images. For simplicity, we did not use the whole dataset but created a fixed reproducible subset in order to speed up the computations while preserving the difficulty due to multiple classes.

The size of subsets used within this project is given below.

- **Training set: 10k samples**
- **Validation set: 2k samples**
- **Test set: 2k samples**

For reproducibility, the data split was performed using a fixed random seed.

Data augmentation has been applied to the training data in order to make the model more robust to various transformations of the input data. Specifically, the following augmentation procedures have been used:

- **Random cropping with padding of 4 pixels.**
- **Random horizontal flip.**

As for the validation and testing data, no augmentation but deterministic preprocessing only has been applied.

The normalization procedure used all the images of the CIFAR-100 training dataset. It is done according to the following parameters:-

- Mean value: (0.5071, 0.4867, 0.4408)
- Standard deviation: (0.2675, 0.2565, 0.2761).

Furthermore, we created two versions of training datasets: augmented for the optimization process and clean one for evaluation purposes.**Model Architectures**

The two models have been designed to incorporate three scales and three exits.

•**ScratchMSDNet (Custom Model):** This is a custom design where the entire model was developed from scratch using primitive PyTorch modules like ConvBNReLU and BottleneckConv layers. This architecture has been designed to mirror the multi-scale and dense connections of an MSDNet. It has a total of 2.29M trainable parameters in its last exit.

•**Official MSDNet (MSDNet-PyTorch):** This model uses official MSDNet class and its corresponding primitives available at kalviny/MSDNet-PyTorch. It has a total of 1.14M trainable parameters in its last exit.

3.2 Model Architecture

Both models were designed to support three scales and three classifier exits for prediction across various layers of the network.

ScratchMSDNet (Custom Model):-

The first model represents an original implementation of an MSDNet-like design created from scratch in PyTorch using such primitive blocks as:

- ConvBNReLU
- BottleneckConv

The model was created with an idea to replicate such features of MSDNet as:

- multi-scale feature propagation
- dense connectivity
- classifier exits

The final classifier of the model comprises approximately 2.29 million trainable parameters.

Official MSDNet (MSDNet-PyTorch):-

The second model represents an original implementation of an MSDNet-like design using an MSDNet class of the official implementation code from kalviny/MSDNet-PyTorch repository. As in the case of the previous model, it is not constructed by hand, but rather using predefined primitives.

In contrast to ScratchMSDNet, this model is more similar to the implementation from the paper. The number of trainable parameters in its final classifier is equal to approximately 1.14 million.

3.3 Training Configuration

For a direct comparison between the two architectures, training was done with an identical setup in terms of optimization parameters.

Loss Function:-

As MSDNet outputs predictions through multiple exits, the loss function was calculated separately for each exit classifier. To compute the total training loss, the losses of each individual exit were summed, giving equal importance to each exit's contribution.

Optimizer:-

Stochastic Gradient Descent (SGD) with Nesterov momentum was used for training:

- **Learning rate = 0.1**
- **Momentum = 0.9**
- **Weight decay = 1e-4**

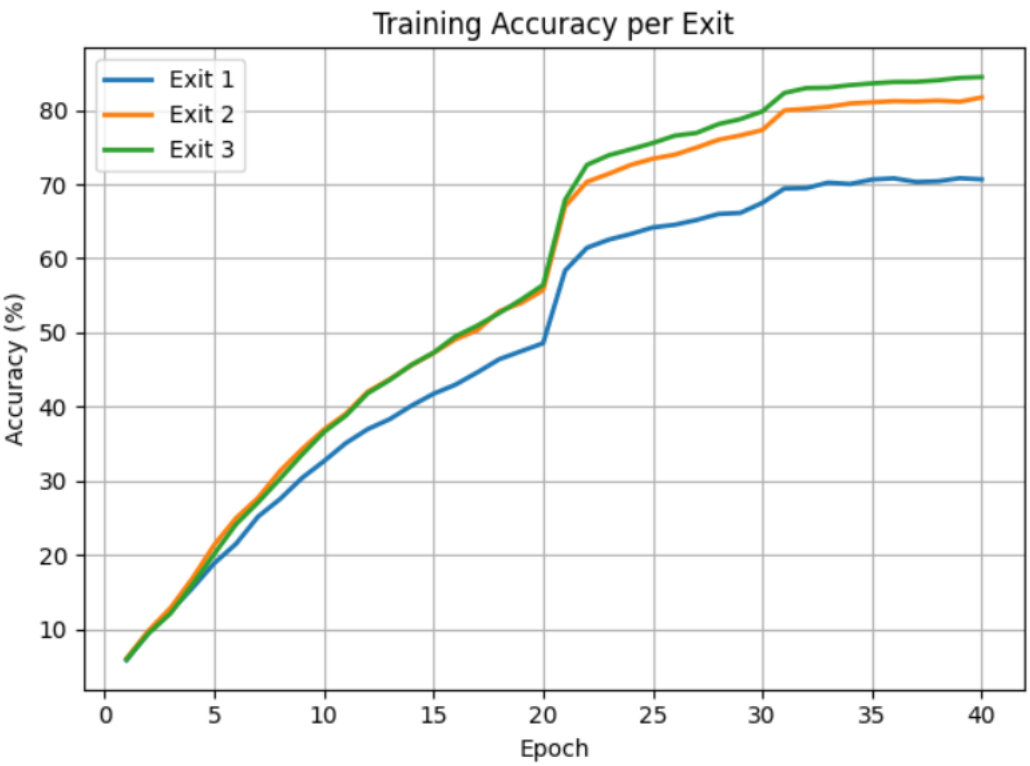
Learning Rate Schedule

To adaptively decrease the learning rate during training, MultiStepLR learning rate schedule was utilized:

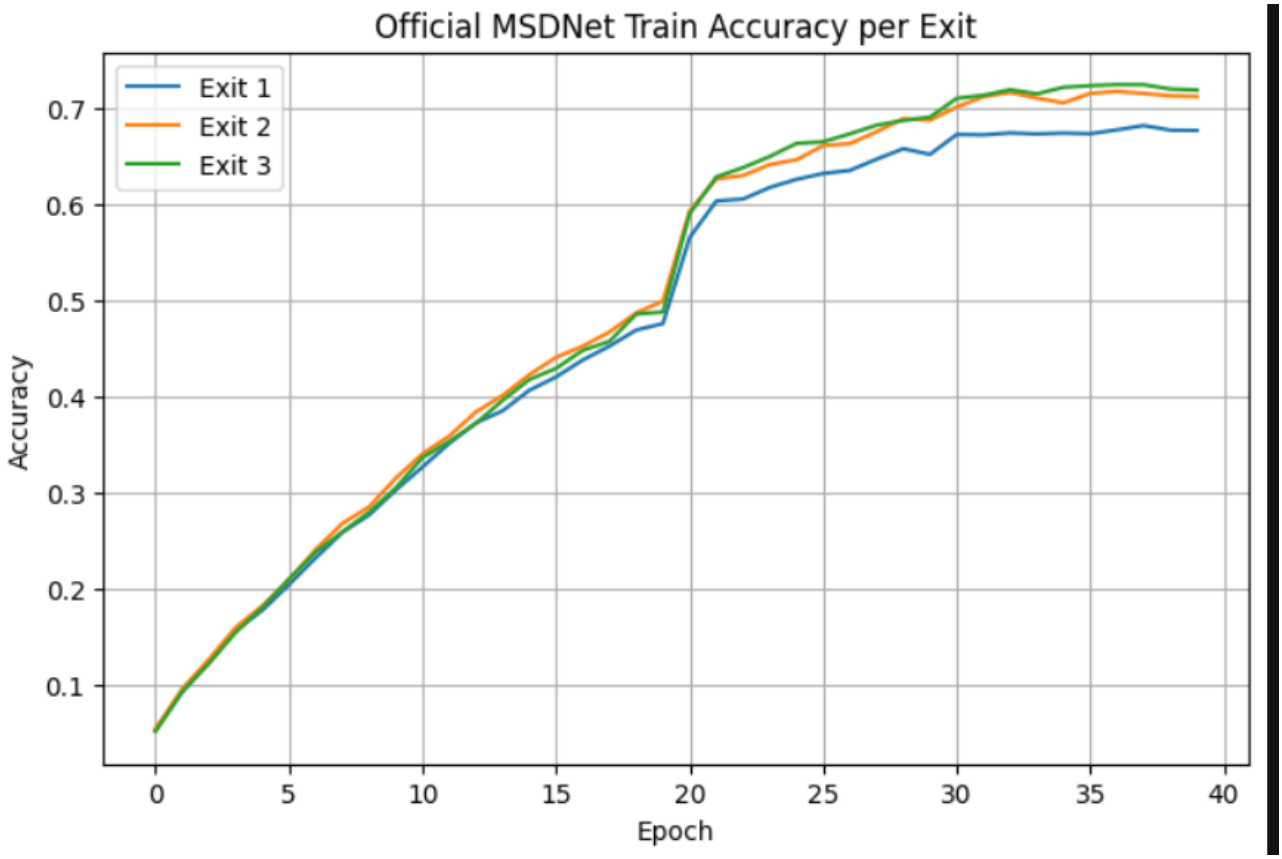
- **learning rate decay at epoch 20**
- **learning rate decay at epoch 30**
- **decay rate (gamma) = 0.1**
- **Training Duration**
- **Epochs = 40**
- **batch size = 64**
- **Evaluation Approach**

During evaluation, performance was calculated at each exit separately to assess the impact of accuracy versus computation time trade-offs. This allowed a direct comparison of the static early-exit behavior of both MSDNet variants.

4.Training Accuracy by Exit:-



Scratch MSDNets training Accuracy Per exist



Official MSDNet training Accuracy Per exist

For the ScratchMSDNet, there is an evident increase in the training accuracy across the three classifier exits through all 40 training epochs. The model begins at a very poor level of training accuracy before gradually learning discriminative features.

It is worth noting that:

- Exit 1 has the poorest training accuracy at any training epoch,
- Exit 2 has higher training accuracy than Exit 1,
- And the highest training accuracy is achieved by Exit 3 in almost all cases.

By the end of the training process, the training accuracy becomes:

- Exit 1: ~71%,
- Exit 2: ~82%, and
- Exit 3: ~85%.

These findings indicate that, due to its ability to learn more discriminative features compared to shallower classifiers, the deepest classifier achieves high training accuracy. This difference is more pronounced as the training progresses, especially from around epoch 20.

There is a significant spike in the training accuracy levels of all classifier exits, specifically in Epoch 21. This spike is attributed to the effect of the learning dynamics during the training following the scheduled optimization phase.

Official MSDNet

The **Official MSDNet** also shows stable learning behavior across all exits. Similar to the scratch implementation, all three exits improve consistently:

- Exit 1 has the poorest training accuracy at any training epoch,
- Exit 2 has higher training accuracy than Exit 1,
- And the highest training accuracy is achieved by Exit 3 and it is more likely similar to Exist 2

However, compared to the scratch model, the gap between exits is **smaller**. The final training accuracies are approximately:

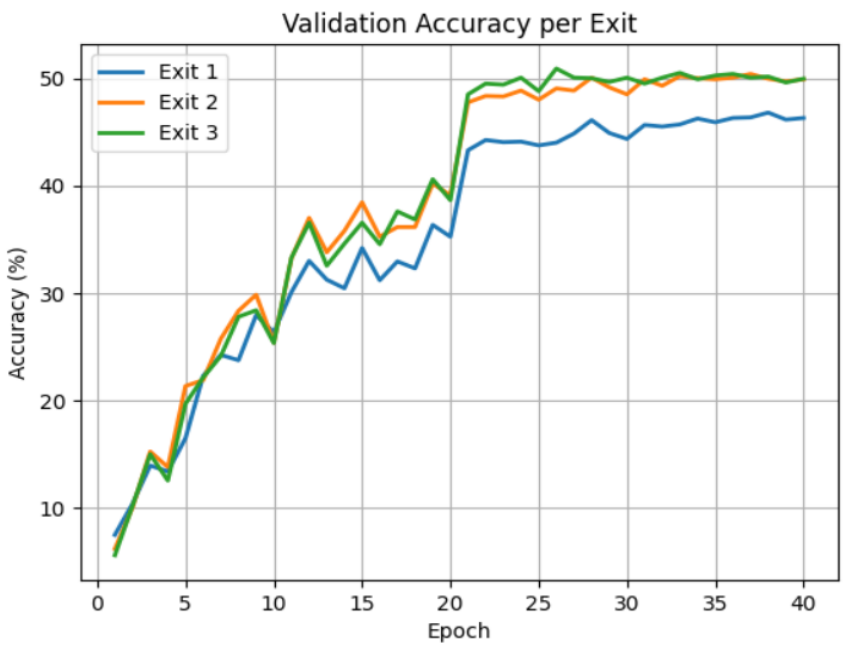
- **Exit 1:** ~68%
- **Exit 2:** ~71.5%
- **Exit 3:** ~72.6%

This indicates that the official model learns useful representations **earlier in the network**, causing the three exits to perform more similarly during training. In other words, the model appears to distribute predictive capability more evenly across the exits.

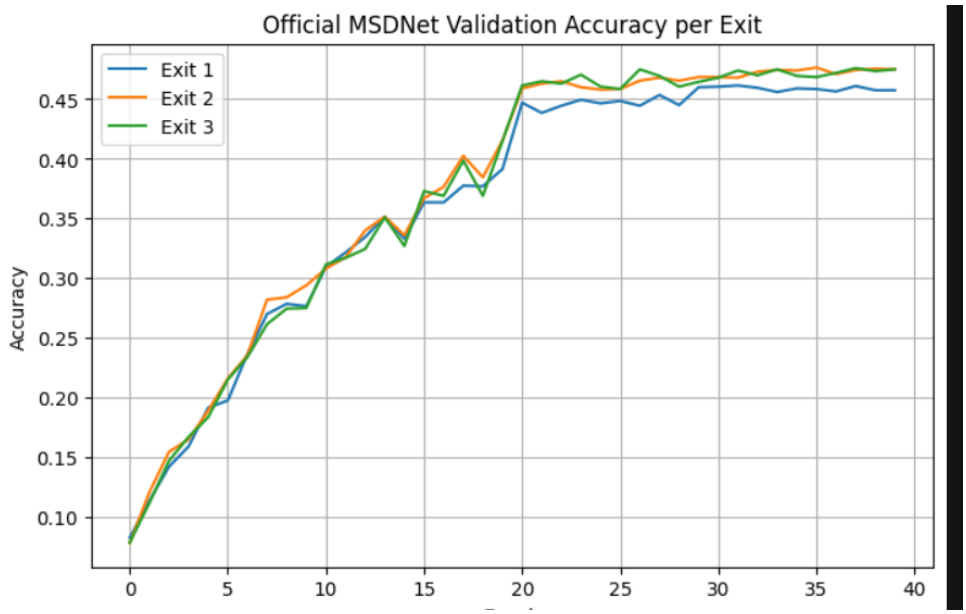
As in the scratch implementation, a noticeable improvement occurs around **epoch 20–21**, after which the training curves begin to flatten and gradually converge.

The relatively small separation between Exit 1, Exit 2, and Exit 3 suggests that the **official implementation is more computationally balanced**, with earlier exits already becoming reasonably strong without requiring as much additional depth.

Validation:-



Scratch MSDNet model validation accuracy per exist



Official MSDNet model validation accuracy per exit

However, in the case of the ScratchMSDNet, improvement can be seen at every exit throughout training time. In this case, a clear distinction is evident among the three exits, with Exit 3 being the highest-performing exit, Exit 2 second-best, while Exit 1 is the least performing one. By the time training ends, the estimated accuracies at each exit are about 46%, 49.05%, and 50.5%, respectively(for best model). This shows that scratch training models gain more advantages from more computations for Exits 1, 2, and 3, respectively(for best model).

In the case of the Official MSDNet, consistent improvement is still evident across all exits. However, the disparity between each exit in terms of performance is significantly low. At the end of training, the estimated accuracies at each exit will be 45.5%, 47%, and 47.56% for Exits 1, 2, and 3, respectively(for best model).

Test Performance:-

The test performance of both models was evaluated independently at each exit.

ScratchMSDNet

- **Exit 1:** 42.65%
- **Exit 2:** 47.35%
- **Exit 3:** 49.10%
- **Test Loss:** 6.6448

The model shows a clear improvement in accuracy with increasing depth, indicating that deeper exits provide more discriminative representations.

Official MSDNet

- **Exit 1:** 44.30%
- **Exit 2:** 44.95%
- **Exit 3:** 45.10%
- **Test Loss:** 6.9893

The accuracy remains relatively consistent across exits, suggesting that useful features are learned early in the network.

Similarity Between Both Models

Both ScratchMSDNet and Official MSDNet show the same overall behavior:

- Accuracy increases with depth (Exit 1 → Exit 3)
- All exits are able to make valid predictions independently
- The models follow the early-exit design principle of MSDNet
- Intermediate classifiers are successfully trained jointly.

Key Difference

- ScratchMSDNet: large improvement with depth
- Official MSDNet: small improvement, more uniform

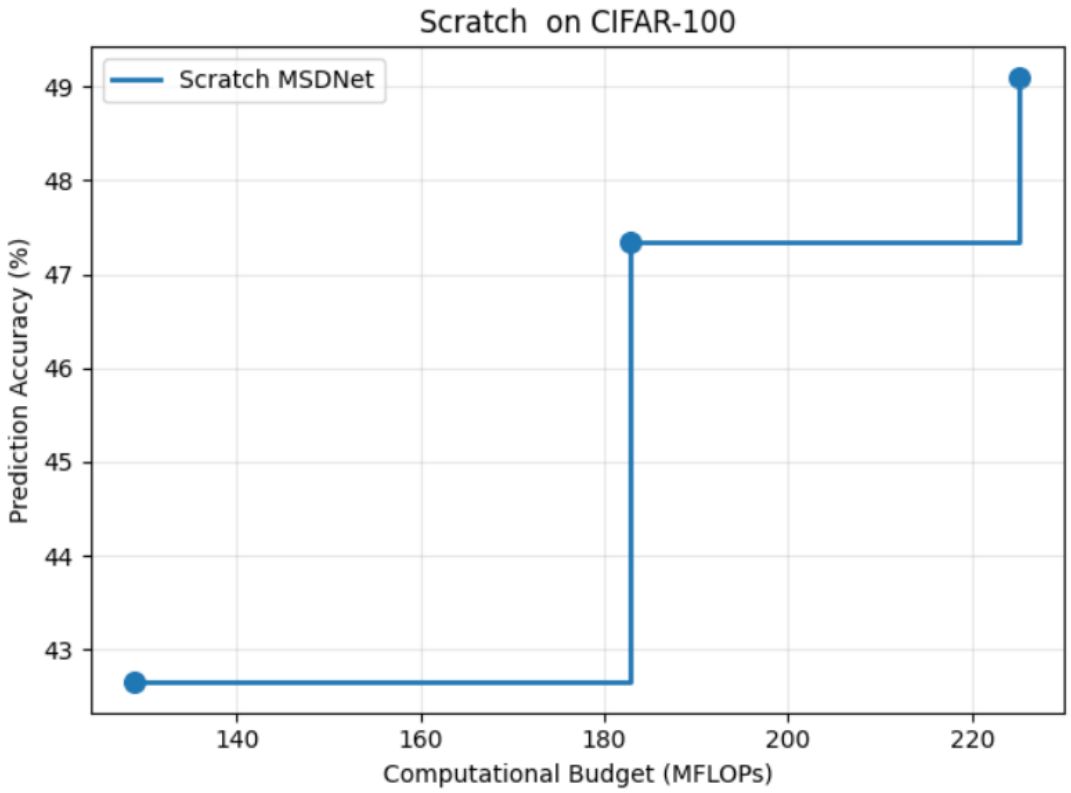
Dynamic prediction vs computational budget

For this analysis, any exit from each classifier is considered as a valid prediction point, and the test accuracy at this exit is plotted against the computation cost (MFLOPs) needed to reach this exit.

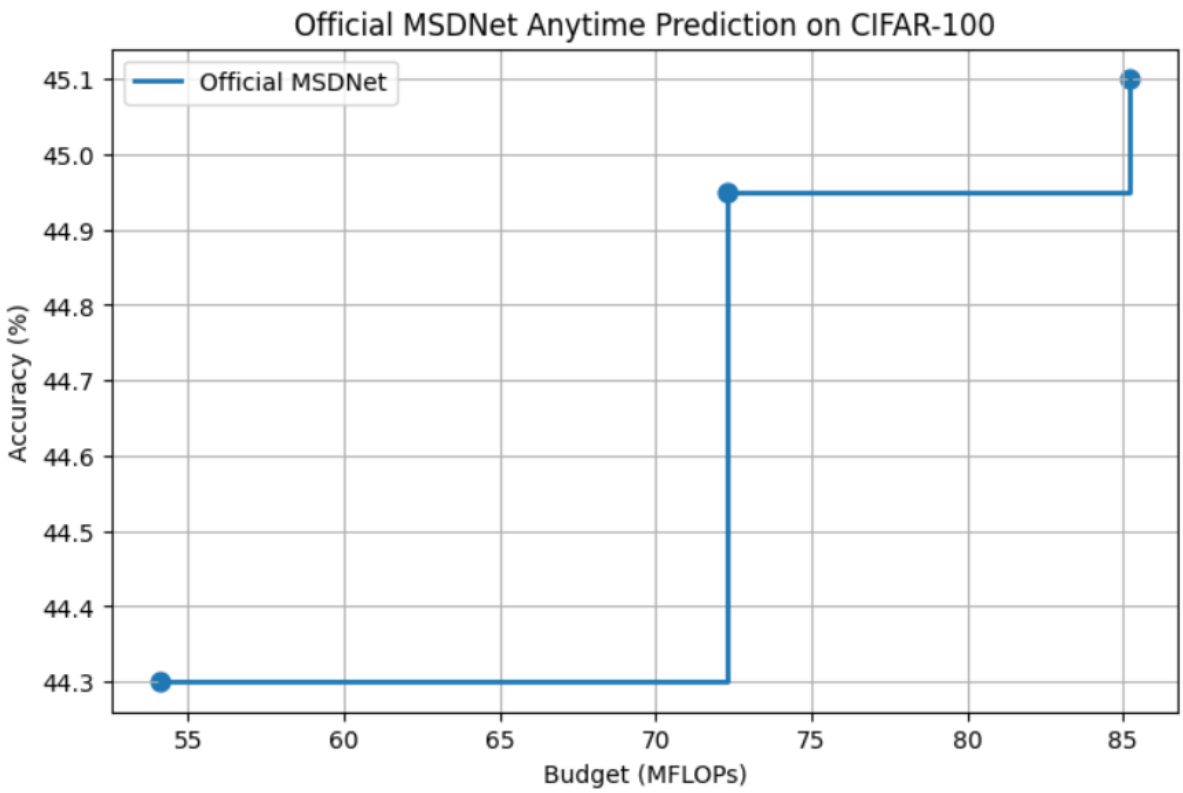
Regarding the Scratch **MSDNet,** the accuracy of prediction is increased **from 42.65% with 128.96 MFLOPs to 47.35% with 182.78 MFLOPs, and further to 49.10% with 225.22 MFLOPs**. Therefore, it is observed that the model requires considerable additional computations, where the predictions made by deeper exits become more accurate.

With respect to the Official MSDNet, the accuracy of prediction is increased from **44.30% with 54.11 MFLOPs to 44.95% with 72.28 MFLOPs, and then to 45.10% with 85.20 MFLOPs**. In comparison with the scratch model, the official one.

Overall, the results illustrate the core **accuracy–computation trade-off** of MSDNet. The **ScratchMSDNet** achieves higher final accuracy, whereas the **Official MSDNet** is substantially more efficient and provides useful predictions at much lower budgets.



Official MSDNet model dynamic prediction vs computational budget



Official MSDNet model dynamic prediction vs computational budget

Similarity:-

Both **ScratchMSDNet** and **Official MSDNet** exhibit the same fundamental **early-exit behavior**. In both models:

- prediction accuracy improves with increasing computational depth,
- all intermediate exits are capable of producing valid predictions,
- and the models demonstrate the core **anytime prediction property** of MSDNet.

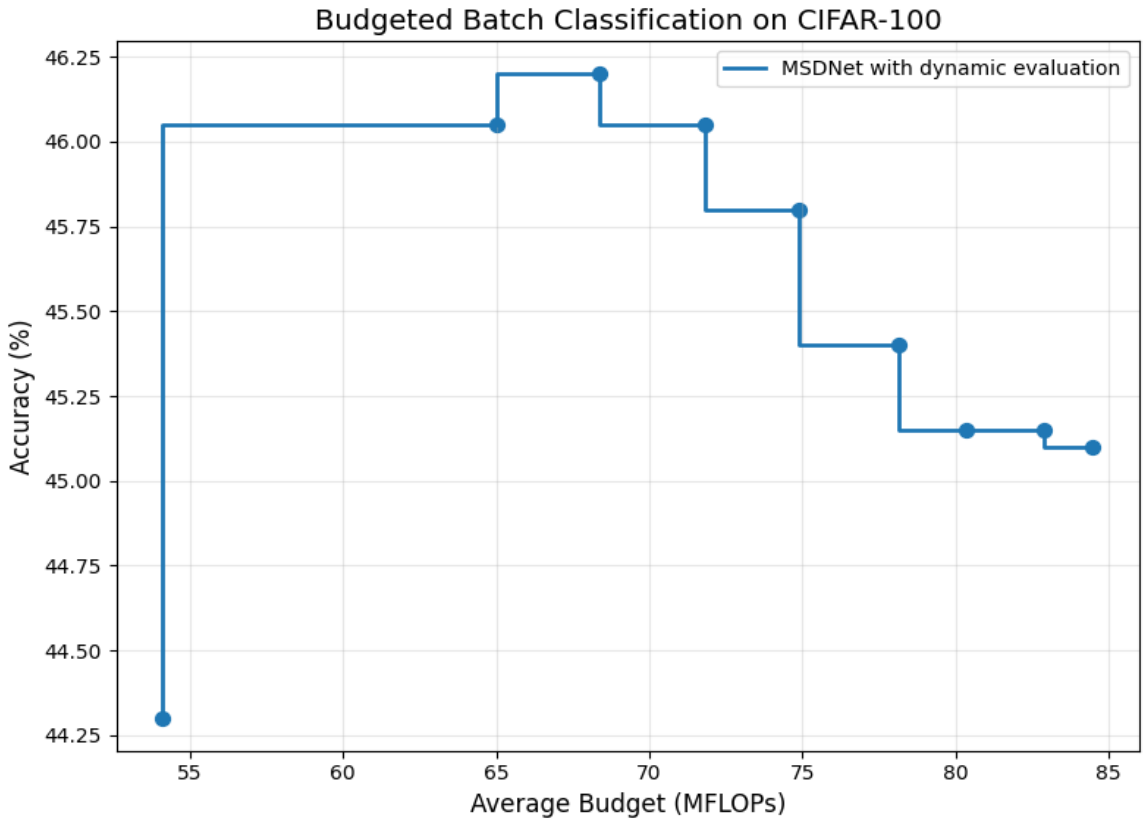
This confirms that both implementations successfully capture the main principle of the original MSDNet architecture:
more computation leads to better prediction quality.

Key Differences:-

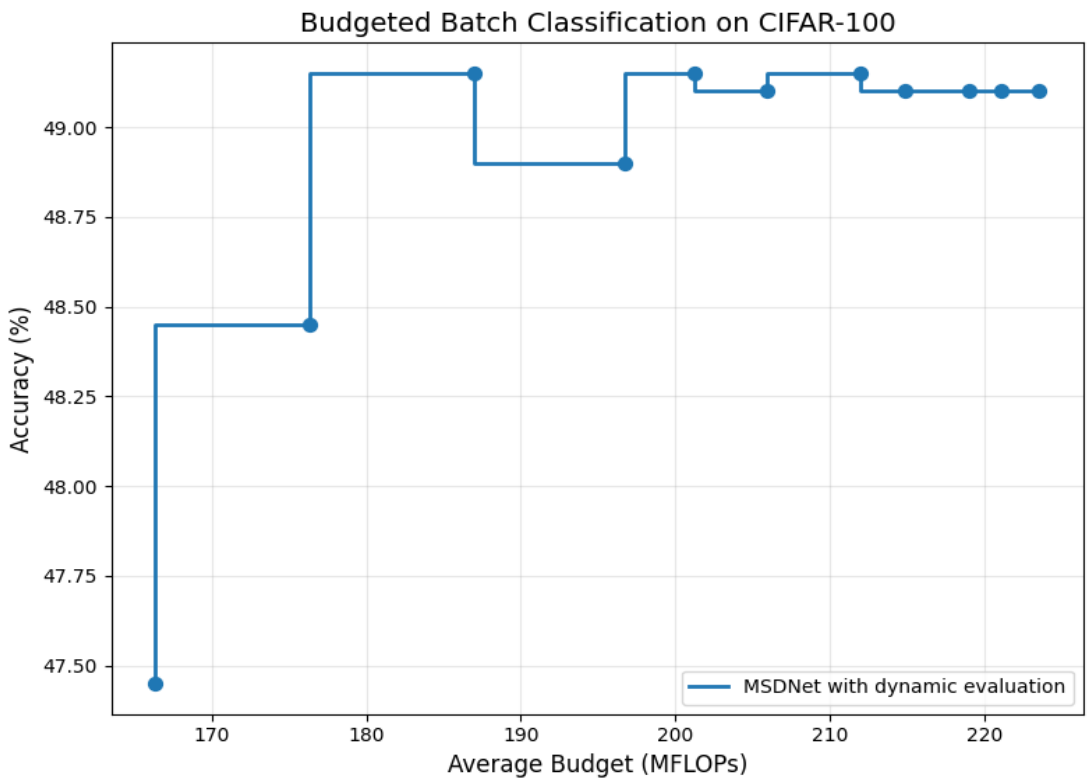
The main difference lies in the **accuracy–efficiency trade-off**.

- The **ScratchMSDNet** achieves **higher final accuracy** (**49.10%**) but requires substantially more computation (**225.22 MFLOPs**).
- The **Official MSDNet** achieves slightly lower final accuracy (**45.10%**) but is significantly more efficient, requiring only **85.20 MFLOPs**.

Budget Based Classsifiation



Official MSDNet model budget based classification graph



Scratch MSDNet model budget based classification graph

The MSDNet is used to see how well it can do its job when it does not have a lot of power. This is different from deep networks that always use the same amount of power to make decisions. The MSDNet can change how power it uses to make decisions for each sample depending on how hard the sample is.

The results show that both the official MSDNet and the one made from scratch can do this. They both use an amount of power to make decisions depending on how much power is available and what the exit policy is. This shows that both models can work with a budget, which's what the MSDNet is designed to do.

The official MSDNet uses a small amount of power around 54 to 85 MFLOPs and it gets an accuracy of 44.3 to 46.2 percent. This means the official model is very good at using power because it can get good results without using a lot of power. The model uses its architecture and the classifiers in the middle to make predictions early without having to use the whole network.

The scratch MSDNet is different. It uses a lot power, around 166 to 223 MFLOPs and it gets an accuracy of 47.4 to 49.1 percent. This means the scratch model is trying to get the results possible by using more power to make decisions.. This also means it uses more power so it is not as efficient, as the official model..

Similarity Analysis

Despite the differences in absolute accuracy and computational cost, both implementations share several key characteristics that confirm the correctness of the scratch model:

- Both models employ a **multi-exit architecture**, where intermediate classifiers are attached at different depths of the network.
- Both demonstrate **adaptive computation**, allowing the inference process to terminate early for easier samples while allocating more computation to harder samples.
- Both exhibit a clear **accuracy–FLOPs trade-off**, where performance varies depending on the available computational budget.
- Both follow the same fundamental principle of **budgeted inference**, which is central to the MSDNet design.

Discussion

What Worked

- **The multi-exit MSDNet behavior was implemented correctly.**
  Both the **ScratchMSDNet** and **Official MSDNet** successfully produce predictions at **three intermediate exits**, confirming correct implementation of the early-exit design.
- **Training converged stably for both models.**
  The training and validation accuracy curves show smooth improvement across epochs, indicating that the optimization setup was effective.
- **Exit-wise learning is clearly visible.**
  In both models, deeper exits achieve higher accuracy than earlier exits, showing that additional depth improves feature quality and classification performance.
- **Static early-exit evaluation was successfully performed.**
  Test accuracy and FLOPs were measured independently for each exit, allowing a clear analysis of the **accuracy–computation trade-off**.
- **Anytime prediction behavior was correctly reproduced.**
  The prediction accuracy versus computational budget plots confirm that both models follow the expected MSDNet principle: **more computation leads to better predictions**.

Challenges and Limitations

- **Implementing the scratch architecture was more complex than the official model.**
  Designing multi-scale blocks, dense feature reuse, and multiple classifier exits manually required careful architectural handling.
- **The ScratchMSDNet is computationally expensive.**

  Although it achieves higher final accuracy, it requires significantly more FLOPs than the official implementation, making it less efficient.
- **The Official MSDNet shows limited gain across exits.**
  While computationally efficient, the official model exhibits only a small improvement from Exit 1 to Exit 3, suggesting that later exits contribute relatively little additional accuracy.
- **A generalization gap is visible in the scratch model.**
  The training accuracy of the ScratchMSDNet increases much more strongly than its validation accuracy, indicating some degree of overfitting.
- **CIFAR-100 remains a challenging benchmark.**
  Since the experiments were performed on a reduced subset of CIFAR-100, the

achieved accuracies should be interpreted as implementation-level results rather than fully optimized benchmark performance.

Overall Observation

- The **ScratchMSDNet** is more **accuracy-oriented**, showing stronger gains from deeper computation.
- The **Official MSDNet** is more **computationally efficient**, with more balanced performance across exits.
- Both implementations successfully demonstrate the core MSDNet principle of **resource-aware prediction through intermediate exits**.